

# The Same Class, in C++

**Read the main notes first.** This sheet does not re-teach the topic. It assumes you already have the Java picture in your head — the stack frame, the heap, the arrow from a reference to an object — and shows you only **what changes when the language is C++**. Every claim here was compiled with `g++ -std=c++17` and run; the outputs shown are real.

★ **SAME** — carries over from Java    ⚠ **DIFFERENT** — C++ does not work like Java here    ► **NOTE** — clarification

◆ **INTERVIEW** — where it shows up

## 0 · The three questions, answered in one table

| Your question                                | C++      | The short answer                                                                                                                                                                                  |
|----------------------------------------------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Do we have the same concept of a call stack? | Yes      | Identical, and more literal than in Java. One frame per call, pushed on call, popped on return, <code>main</code> at the bottom. The diagram from class is unchanged.                             |
| Do we have the same concept of heap memory?  | Yes, but | The heap exists and works the same way — but in C++ <b>you</b> decide what goes there, and <b>you</b> free it. There is no garbage collector. And unlike Java, objects can live on the stack too. |
| Is passing a parameter still by value?       | Yes, but | By default yes — but the copy is the <b>whole object</b> , not just an address. C++ also gives you real <i>pass-by-reference</i> with <code>&amp;</code> , which Java has no equivalent for.      |

## 1 · Call stack — unchanged nothing new to learn

The three functions from class, in C++:

```
int add(int x, int y)    { return x + y; }
int product(int x, int y) { return x * y; }
int subtract(int x, int y) { return x - y; }

int main() {
    int x = 10, y = 20;
    int t1 = add(x, y);           // 30
    int t2 = product(x, y);       // 200
    int t3 = subtract(x, y);      // -10
    cout << t1 + t2 + t3;        // 220
}
```

★ **SAME AS JAVA**

Same frames, same order: `main` pushed first and popped last; `add` pushed, popped, then `product`, then `subtract`. The crossed-out-boxes diagram from the class notes is correct for C++ exactly as drawn. Locals still die when the frame is popped.

★ **SAME**

Output is `220`, verified. Nothing about the call stack changes between the two languages.

⚠ **DIFFERENT**

Run out of stack in Java and you get a catchable `StackOverflowError`. In C++ you get **undefined behaviour** — usually a segmentation fault, with no exception and no message. Debugging it is harder, so infinite recursion is more expensive to get wrong.

► **NOTE**

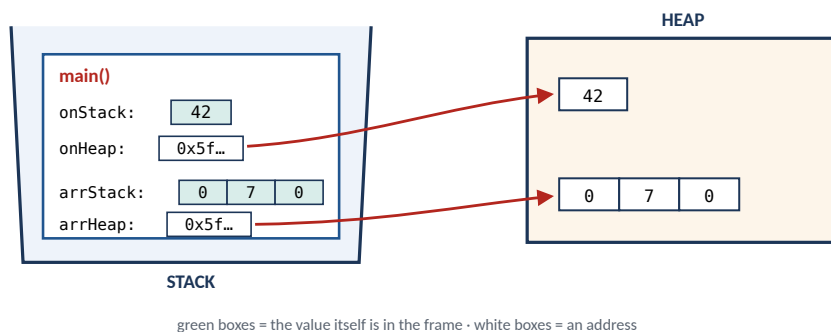
C++ frames are usually *bigger*, because objects themselves can sit inside them — see the next section. Java frames only ever hold primitives and addresses.

In Java the rule was simple and had no exceptions: **primitives and references on the stack, every object on the heap**. You had no say in it — `new` was the only way to make an object, and `new` always meant heap.

C++ removes that rule. **You choose**, per variable:

```
int onStack = 42;           // the 42 is INSIDE main's frame
int *onHeap = new int(42); // the 42 is in the heap;
                          // only the pointer is in the frame

int arrStack[3] = {0,0,0}; // all 3 slots are IN the frame
int *arrHeap = new int[3](); // the array is in the heap
```



## No garbage collector — this is on you

Recall Example 3 from class: `ar` was re-pointed at a new array, the old array became unreachable, and the **garbage collector removed it**. Here is the same code in C++:

```
int *ar = new int[3];
ar[1] = 9;
ar[2] = 5;

ar = new int[5];           // the old array is now unreachable...
                          // ...and NOBODY frees it. Memory leak.
```

### △ DIFFERENT — THE WHOLE DIAGRAM CHANGES AT THE LAST STEP

Everything up to the last line is identical to the Java notes. The old 3-element array is orphaned in the heap exactly as before. But there is no garbage collector, so it just **sits there for the life of the program**. You had to write `delete[] ar`; *before* re-pointing.

### △ DIFFERENT

In Java, `int[] ar = new int[3]` ALWAYS puts the array on the heap. In C++, `int arr[3]` puts it on the stack. Same-looking declaration, different memory region.

### ★ SAME

The heap itself behaves identically: it outlives the frame that created it, it is reached through an address, and two pointers holding the same address see the same object.

### ► THE TRANSLATION

Java's **reference** is C++'s **pointer**. Same picture, same arrow. C++ just lets you see and print the number.

### ◆ INTERVIEW

"Where does this object live?" is a real C++ question with a real answer. In Java it is not a question at all — the answer is always the heap.

### △ TWO FAILURE MODES JAVA DOES NOT HAVE

**Leak** — you forget `delete`, memory is never returned. **Dangling pointer** — you `delete` and then use the pointer anyway, reading memory that is no longer yours.

### ► IN PRACTICE

Modern C++ mostly stops writing raw `new/delete`. You use `vector`, `string`, and smart pointers, which free themselves when their frame pops. That is the closest C++ gets to a GC — and it is deterministic, which the GC is not.

### ◆ INTERVIEW

"What is a memory leak, and how do you avoid one?" — the answer is this section: every `new` needs exactly one matching `delete`, or don't write `new` at all.

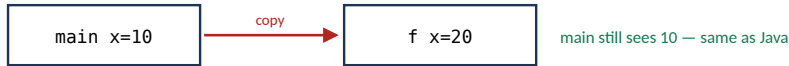
| What you allocated                | What frees it                                                  |
|-----------------------------------|----------------------------------------------------------------|
| <code>new int(42)</code>          | <code>delete p;</code>                                         |
| <code>new int[3]</code>           | <code>delete[] p;</code> — the <code>[]</code> is not optional |
| <code>int arr[3];</code> (stack)  | nothing — the frame pops and it is gone                        |
| <code>vector&lt;int&gt; v;</code> | nothing — the vector frees its own heap buffer                 |

### 3 · How a method call passes its parameters

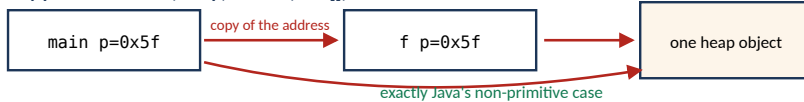
three modes, not one

In Java there was exactly one mechanism — **copy the value** — and the only variation was whether the value happened to be a number or an address. C++ keeps that mechanism as the default and adds a second one.

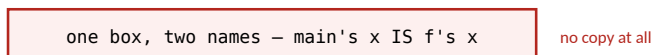
#### 1 · by value — `void f(int x)`



#### 2 · by pointer — `void f(int* p) / void f(int a[])`



#### 3 · by reference — `void f(int& x)` ← no Java equivalent



#### ★ SAME

Modes 1 and 2 are precisely the two cases from class — Case 1 (primitive) and Case 2 (array). If you understood those, you already understand two thirds of C++.

#### ⚠ DIFFERENT

Mode 3 is the genuinely new thing. `int& x` does not make a copy; it makes `x` another name for the caller's variable. Java cannot express this — which is why the "predict the output" question has only one possible answer in Java, and two in C++.

#### ► HOW TO READ A SIGNATURE

No `*` and no `&` → a copy is made. `*` → an address is copied. `&` → no copy; you are working on the caller's object.

### The trap Java people fall into

```
void funcVec(vector<int> v) { v[1] = 20; } // by value
void funcVecRef(vector<int> &v) { v[1] = 20; } // by reference

vector<int> v1 = {10,10,10};
funcVec(v1);      cout << v1[1]; // 10 ← the copy changed

vector<int> v2 = {10,10,10};
funcVecRef(v2);   cout << v2[1]; // 20 ← v2 changed
```

#### ⚠ DIFFERENT — AND THIS ONE COSTS YOU MARKS AND MILLISECONDS

A Java programmer reads `funcVec(v1)` and expects 20, because in Java passing a container always shares the object. In C++ it prints **10**: `vector<int> v` as a parameter **copies every element**. On a vector of  $10^5$  elements that is a full copy on every call — a common, silent cause of TLE.

#### ◆ INTERVIEW

"Why did your solution TLE?" — passing a large container by value inside a loop is one of the top three answers in C++.

#### ► RULE OF THUMB

Pass small things (`int`, `char`, `double`) by value. Pass big things (`vector`, `string`, structs) by `const&` if you only read, by `&` if you write.

#### ► WHY RAW ARRAYS DIFFER

`void f(int a[])` looks like it takes an array, but C++ silently turns it into `int* a`. Nothing is copied except the address — which is why the array case behaves like Java and the vector case does not.

Which is why competitive C++ is full of `const vector<int>&` and `vector<int>&`: the `&` is there to *stop the copy*.

## "Predict the output", in C++ — now it has two answers

```
// A · the pointer is passed BY VALUE — the Java case
void fun(int *a) { a = new int[1]; a[0] = 100; }

// B · the pointer is passed BY REFERENCE — impossible in Java
void funRefPtr(int *&a) { a = new int[1]; a[0] = 100; }

int *a = new int[3]{10, 20, 30};
fun(a);          cout << a[0];    // 10   ← same as Java
funRefPtr(a);    cout << a[0];    // 100   ← impossible in Java
```

### ★ THE POINT OF THIS COMPARISON

In version **A**, fun got a copy of the address and overwrote its own copy — main's arrow never moved, so main still prints 10. That is exactly the reasoning from class, unchanged.

In version **B**, a is a reference to main's pointer. There is no copy, so re-pointing inside the function re-points main's variable too, and main prints 100. **The rule "replacing the arrow is never visible to the caller" is a rule about pass-by-value, not a law of nature** — Java just never gives you the other option.

### ⚠ ALSO NOTE

Version A leaks: the `new int[1]` made inside `fun` becomes unreachable the moment `fun` returns, and nothing frees it. In the Java version the GC collected it silently.

### ► DO THIS

Write version A, run it under `valgrind` or just reason it out: how many bytes leak per call? Then fix it with a `delete[]`.

## 4 · Java → C++ in one table

| Idea                        | Java (from class)                                   | C++                                                                  |
|-----------------------------|-----------------------------------------------------|----------------------------------------------------------------------|
| Call stack                  | One frame per call, LIFO                            | Identical                                                            |
| Stack exhausted             | <code>StackOverflowError</code> , catchable         | Undefined behaviour — usually a segfault                             |
| What's in a frame           | Primitives + references only                        | Primitives, pointers, <b>and whole objects</b>                       |
| Where objects live          | Always the heap                                     | Your choice: <code>Obj o;</code> stack · <code>new Obj()</code> heap |
| Reference to an object      | A reference (address you can't print)               | <b>A pointer — same thing, and you can print it</b>                  |
| Freeing heap memory         | Garbage collector, automatic                        | <code>delete</code> / <code>delete[]</code> , or RAII. Yours to do.  |
| Passing an <code>int</code> | Copy of the value                                   | <b>Copy of the value</b>                                             |
| Passing an array            | Copy of the reference → shared                      | <b>Decays to a pointer → shared</b>                                  |
| Passing a container         | <code>ArrayList</code> : copy of reference → shared | <code>vector&lt;int&gt;</code> : <b>full copy of every element</b>   |
| True pass by reference      | Does not exist                                      | <code>void f(int&amp; x)</code> — no copy at all                     |
| Reassign the parameter      | Never visible to the caller                         | Not visible — <i>unless</i> the parameter is a reference             |

**Verification.** Every C++ snippet in this handout is from one program compiled with `g++ -std=c++17` and executed; the outputs shown (`220 · 10 · 20 · 20 · 10 · 20 · 10 · 100 · 100`) are its real output, not expected values. Source: `notes_memory_management_cpp_companion_2026-09-01.cpp`.